VGU COLLEGE · JAVA · MODULE III

Java Collections Framework

The Complete Picture: List · Set · Queue · Map

OUR STORY

Our story for the whole talk: the students of section **CSE-A**

Roll no	Name	Total
101	Hita	438
102	Sakshi	244
103	Somya	465
104	Ritika	412
105	Disha	438

How to read this deck: points marked *** are the ones students must remember.

Contents

1 Why Do We Need Collections? 3
2 Definitions 3
3 Three Parts of the Framework 3
4 Array vs Collection 4
5 The Big Map: Collection Hierarchy 4
6 The Big Map: Map Hierarchy 5
7 Four Families, Four Rules 5
8 Interface on the Left, Class on the Right 6
9 Generics and Wrapper Classes 6
10 The `Collection` Interface: Methods Everyone Has 7

PART A · LIST

11 List: Definition and Features 8
12 List Methods 8
13 ArrayList: Definition 9
14 ArrayList: How It Works Inside 9
15 LinkedList: Definition 10
16 LinkedList: How It Works Inside 10
17 ArrayList vs LinkedList 11
18 Vector: Definition 11
19 ArrayList vs Vector 12
20 Stack: Definition 12
21 Stack Methods 13
22 The Whole List Family on One Slide 13

PART B · SET

23 Set: Definition and Features 14
24 Three Sets, Same Sign-up Sheet 14
25 Hashing: How HashSet Finds Duplicates 15
26 TreeSet: Sorted Set 15
27 Set Comparison 16

PART C · QUEUE AND DEQUE

28 Queue: Definition 17
29 Queue Methods: Two Sets 17
30 PriorityQueue 18
31 Deque and ArrayDeque 18
32 Queue Family Comparison 19

PART D · MAP

33 Map: Definition 20
34 Map Methods 20
35 How HashMap Works Inside 21
36 HashMap, LinkedHashMap, TreeMap, Hashtable 21

PART E · WORKING WITH COLLECTIONS

37 Iterator: Visiting Elements 22
38 Fail-Fast vs Fail-Safe 23
39 Sorting: Comparable vs Comparator 24
40 The `Collections` Utility Class 25
41 Immutable Collections 25

PART F · THE FULL PICTURE

42 Master Table: Every Class at a Glance 26
43 Speed Table (Big-O) 26
44 Legacy Classes 27
45 Which Collection Do I Use? (Decision Map) 27
46 Top Remember Points (***) 28
47 Quick Check 28
Thank You 29

1. Why Do We Need Collections?

Module II kept every member in an array: `CollegeMember[] members = new CollegeMember[60];`

The college says...	With an array
"61st student took admission"	❌ Array is full, size is fixed
"Sakshi moved to section B"	❌ Shift everyone after her by hand
"Is Somya in this section?"	❌ Write a loop
"Show me roll no 103"	❌ Write a loop
"Give me the merit list"	❌ Write our own sorting code
"Hita signed up twice"	❌ Write code to stop duplicates

*** REMEMBER

We were writing **storage code** again and again instead of **college code**. Collections give us ready-made storage.

2. Definitions

DEFINITION

Collection

An object that stores a group of objects (called **elements**) as a single unit.

DEFINITION

Framework

A ready-made set of interfaces and classes that gives a standard way to solve a problem.

DEFINITION

Collections Framework

A set of interfaces and classes in the `java.util` package used to **store** and **work on** a group of objects (add, remove, search, sort).

*** REMEMBER

Everything in this deck lives in `java.util` (except `Iterable` and `Comparable`, which are in `java.lang`).

3. Three Parts of the Framework

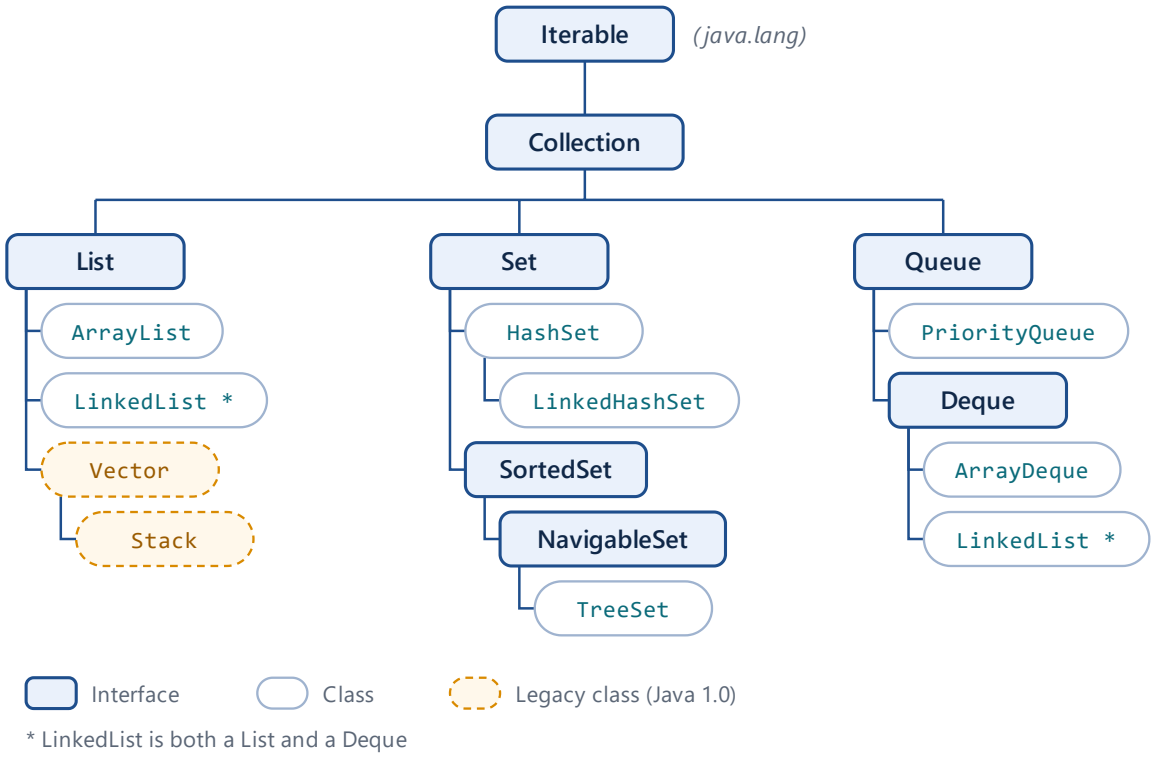
Part	Meaning	Examples
Interfaces	WHAT a container can do	`Collection`, `List`, `Set`, `Queue`, `Deque`, `Map`
Classes (implementations)	HOW it is done, ready to use	`ArrayList`, `LinkedList`, `HashSet`, `HashMap` ...
Algorithms	Ready-made helper methods	`Collections.sort()`, `Collections.max()`

4. Array vs Collection

	Array	Collection
Size	Fixed when created	Grows and shrinks by itself
Stores	Primitives **and** objects	**Objects only**
Ready-made methods	None (only `length`)	`add`, `remove`, `contains`, `size` ...
Insert in the middle	Shift by hand	`add(index, x)`
Search / sort	Write it yourself	One method call
Print all	Loop needed	`System.out.println(list)`
Type safety	Yes	Yes, with Generics `<Student>`

> *** REMEMBER
> Array is still fine when the size **never changes** (a student always has 5 marks).

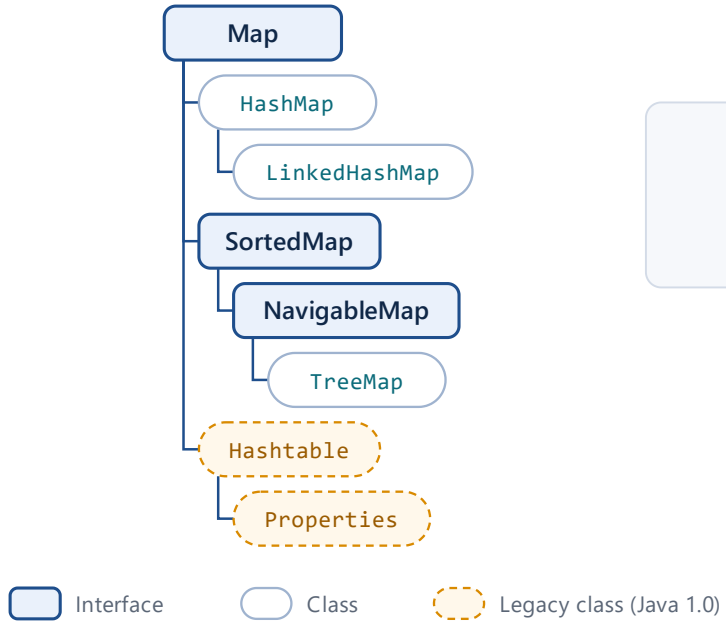
5. The Big Map: Collection Hierarchy



* LinkedList is both a List and a Deque

- **Interfaces:** Iterable, Collection, List, Set, SortedSet, NavigableSet, Queue, Deque
- **Classes:** everything else
- **Legacy (old, Java 1.0):** `Vector`, `Stack`

6. The Big Map: Map Hierarchy



*** REMEMBER

`Map` is **not** a `Collection`. It stores **pairs** (key → value), not single elements.

7. Four Families, Four Rules

Family	Rule	VGU College job	Main classes
List	Ordered, index, duplicates allowed	Section register in roll order	`ArrayList`, `LinkedList`, `Vector`, `Stack`
Set	**No duplicates**	Workshop sign-up: each name once	`HashSet`, `LinkedHashSet`, `TreeSet`
Queue	**First in, first out**	Line at the fee counter	`PriorityQueue`, `ArrayDeque`, `LinkedList`
Map	**Key → value**, unique keys	Roll no → student	`HashMap`, `LinkedHashMap`, `TreeMap`, `Hashtable`

*** REMEMBER

Learn the **rule** of each family first. The class names come after.

8. Interface on the Left, Class on the Right

```
List<Student> section = new ArrayList<>();
//  interface              class
//  (WHAT it can do)       (HOW it does it)
```

- Want a `LinkedList` later? Change **one word** on the right.
- Same idea as Module II: `Payment mode = new UpiPayment(...)` (polymorphism).

> *** REMEMBER
> Program to the **interface**, not the class.

9. Generics and Wrapper Classes

```
List<Student> section = new ArrayList<>();   // only Students allowed
List<Integer> marks = new ArrayList<>();
marks.add(85);          // int → Integer   (autoboxing)
int m = marks.get(0);   // Integer → int   (unboxing)
// List<int> x;         // ✗ not allowed
```

Primitive	Wrapper used in collections
`int`	`Integer`
`double`	`Double`
`char`	`Character`
`boolean`	`Boolean`

- `<Student>` = **Generics**: wrong type → **compile error**, no cast needed when reading.

> *** REMEMBER
> Collections store **objects only**.

10. The `Collection` Interface: Methods Everyone Has

Method	Does
`add(x)`	Add (returns `true` if added)
`remove(x)`	Remove x
`contains(x)`	Is x there? (uses `equals()`)
`size()` / `isEmpty()`	How many? / Empty?
`clear()`	Remove everything
`toArray()`	Collection → array
`iterator()`	Get an Iterator

Bulk methods (`java = [Hita, Somya]`, `python = [Somya, Ritika, Disha]`)

Method	Meaning	Result
`addAll(python)`	Union	`[Hita, Somya, Somya, Ritika, Disha]`
`retainAll(python)`	Common only	`[Somya]`
`removeAll(python)`	Remove those	`[Hita]`
`containsAll(python)`	All present?	`false`

> *** REMEMBER
> `Collection` has **no** `get(index)`. Index is only in `List`.

PART A

List

11. List: Definition and Features

DEFINITION

List

An **ordered** collection that allows **duplicates** and gives access by **index** (0, 1, 2 ...).

- Keeps **insertion order**
- **Duplicates allowed**: `[Hita, Somya, Hita]`
- **Index-based**: `get(i)`, `set(i, x)`, `add(i, x)`, `remove(i)`
- Many `null`s allowed

Class	Inside	Since
`ArrayList`	Resizable array	Java 1.2
`LinkedList`	Doubly linked nodes	Java 1.2
`Vector`	Resizable array, synchronised	Java 1.0 (legacy)
`Stack`	Extends `Vector`, LIFO	Java 1.0 (legacy)

12. List Methods

Method	Does	In our section
`get(i)`	Element at index i	`get(0)` → Hita
`set(i, x)`	Replace at i	Sakshi re-checked
`add(i, x)`	Insert at i, others move right	Ritika joins at index 3
`remove(i)`	Remove at i, others move left	Disha moves to section B
`indexOf(x)` / `lastIndexOf(x)`	First / last position, or -1	Somya → 2
`subList(a, b)`	Part from a to b-1	Front bench: 0, 1
`sort(comp)`	Sort the list	Merit list

*** REMEMBER

Wrong index → `IndexOutOfBoundsException`.

13. ArrayList: Definition

DEFINITION

ArrayList

A List that stores elements in a **resizable (dynamic) array** inside.

```
List<Student> section = new ArrayList<>();
section.add(new Student(101, "Hita", 438));
section.add(new Student(103, "Somya", 465));
section.get(1);                              // Somya, jumps straight there
```

Features

- Fast **read** by index (`get(i)`): jumps directly
- Slow **insert/remove in the middle**: elements must shift
- Not synchronised (not thread-safe), so it is fast
- Allows duplicates and `null`

14. ArrayList: How It Works Inside

Our own version: CustomArrayList.java

```
add("Hita"), add("Sakshi"), add("Somya")

 index:   0       1        2      3    4   ...  9
        [Hita | Sakshi | Somya | -- | -- | ... | -- ]     capacity 10, size 3
```

1. Starts with capacity **10** (on the first `add`).
2. `add()` puts the element at index `size`, then `size++`.
3. Array **full**? → make a **new array 1.5 times bigger** → copy all → continue. `10 → 15 → 22 → 33 ...`
4. `remove(i)` → every element after i **shifts left** (`System.arraycopy`).

*** REMEMBER

ArrayList grows by **1.5x**. Size = elements present, Capacity = space available.

15. LinkedList: Definition

DEFINITION

LinkedList

A List made of **nodes**. Each node stores the **data** plus a link to the **previous** and **next** node (**doubly linked list**).

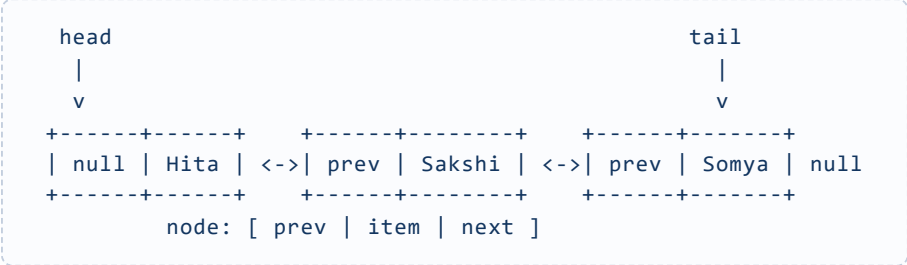
```
LinkedList<String> notices = new LinkedList<>();
notices.add("Fee last date 30 Oct");
notices.addFirst("Java test on Monday");     // new notice on top
notices.addLast("Library closes at 6 pm");
notices.removeFirst();                       // test is over
```

Features

- Fast **add/remove at the ends** (`addFirst` , `removeLast` ...)
- Slow `get(i)` : must walk node by node
- No capacity, no copying: it just links a new node
- Implements **both** `List` and `Deque`

16. LinkedList: How It Works Inside

Our own version: CustomLinkedList.java



Insert Ritika between Hita and Sakshi: no shifting, just change 4 links.



Remove Sakshi: join her neighbours: `Hita.next = Somya` , `Somya.prev = Hita` .

- `get(i)` starts from the **nearer end** (head or tail) and walks.

*** REMEMBER

LinkedList = nodes + links. Adding/removing = **changing links**, not shifting.

17. ArrayList vs LinkedList

	`ArrayList`	`LinkedList`
Inside	Resizable array	Doubly linked nodes
`get(i)`	**Fast** O(1)	Slow O(n), walks
Add/remove at the end	Fast	Fast
Add/remove at the start	Slow (shift all)	**Fast** O(1)
Add/remove in the middle	Slow (shift)	Fast once the place is found
Memory	Less (only data)	More (data + 2 links per node)
Interfaces	`List`, `RandomAccess`	`List`, `Deque`
Use when	Mostly **reading**	Mostly **adding/removing at the ends**

*** REMEMBER

`ArrayList` is the **default choice** for a List.

18. Vector: Definition

DEFINITION

Vector

An old (**legacy**, Java 1.0) List that works like `ArrayList` but every method is **synchronised** (thread-safe).

```
Vector<String> attendance = new Vector<>();
attendance.add("Hita");
attendance.add("Somya");
attendance.addElement("Ritika");          // old method name
attendance.get(0);                        // Hita
attendance.capacity();                    // 10
```

Features

- Resizable array inside, default capacity **10**
- Grows **double** (2x): `10 → 20 → 40`
- **Synchronised** → safe for many threads, but **slower**
- Has the old `Enumeration` (`elements()`) to loop

19. ArrayList vs Vector

	`ArrayList`	`Vector`
Introduced	Java 1.2	Java 1.0 (**legacy**)
Synchronised	No	**Yes**
Speed	**Fast**	Slow (locks every call)
Grows by	**1.5x** (50%)	**2x** (100%)
Loop with	`Iterator`	`Iterator` and `Enumeration`
Use today	Yes	Only in old code

*** REMEMBER

Need thread safety? Use `Collections.synchronizedList()` or `CopyOnWriteArrayList`, not `Vector`.

20. Stack: Definition

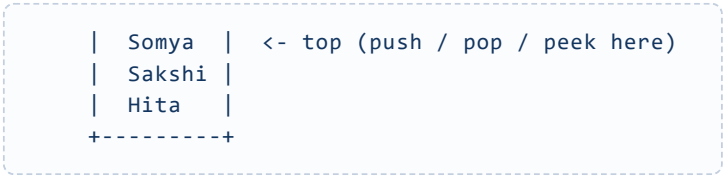
DEFINITION

Stack

A legacy class that **extends** `Vector` and works as **LIFO** (Last In, First Out).

Mr. Verma's pile of answer sheets: the last sheet put on top is checked first.

```
Stack<String> sheets = new Stack<>();
sheets.push("Hita");
sheets.push("Sakshi");
sheets.push("Somya");      // on top
sheets.peek();             // Somya (only looks)
sheets.pop();              // Somya (removes)
sheets.search("Hita");     // 2 (position from top, starts at 1)
sheets.empty();            // false
```



21. Stack Methods

Method	Does	Empty stack?
`push(x)`	Put x on top	-
`pop()`	Remove and return top	`EmptyStackException`
`peek()`	See top, do not remove	`EmptyStackException`
`empty()`	Is the stack empty?	`true`
`search(x)`	Position from top (1-based), or -1	`-1`

Stack vs ArrayDeque

	`Stack`	`ArrayDeque`
Type	Class (extends `Vector`)	Class (implements `Deque`)
Synchronised	Yes (slow)	No (**fast**)
Problem	Also has `get(i)`, `add(i, x)` (breaks LIFO)	Only stack/queue methods

> *** REMEMBER
> For a stack today, use `Deque<String> s = new ArrayDeque<>();` with `push/pop/peek`.

22. The Whole List Family on One Slide

	`ArrayList`	`LinkedList`	`Vector`	`Stack`
Inside	Array	Linked nodes	Array	Array (Vector)
Order	Insertion	Insertion	Insertion	LIFO
Duplicates	Yes	Yes	Yes	Yes
`null`	Yes	Yes	Yes	Yes
Synchronised	No	No	**Yes**	**Yes**
Grows	1.5x	Node by node	2x	2x
Best for	Reading	Ends	Old code	Old code

PART B

Set

23. Set: Definition and Features

DEFINITION

Set

A collection that does **not** allow duplicate elements.

Workshop sign-up sheet: `Somya, Hita, Ritika, Hita, Disha, Somya` → each name **once**.

```
Set<String> names = new HashSet<>();
names.add("Hita");          // true
names.add("Hita");          // false, ignored (no error)
```

- **No duplicates**
- **No index**: there is no `get(i)`
- `add()` returns `false` for a duplicate

Class	Order	Inside
`HashSet`	No fixed order	Hash table (`HashMap`)
`LinkedHashSet`	Insertion order	Hash table + linked list
`TreeSet`	Sorted order	Red-black tree (`TreeMap`)

24. Three Sets, Same Sign-up Sheet

Set	Output
`HashSet`	`[Disha, Ritika, Somya, Hita]` (any order)
`LinkedHashSet`	`[Somya, Hita, Ritika, Disha]` (sign-up order)
`TreeSet`	`[Disha, Hita, Ritika, Somya]` (A to Z)

*** REMEMBER

Fast → `HashSet`. Keep order → `LinkedHashSet`. Sorted → `TreeSet`.

25. Hashing: How HashSet Finds Duplicates

DEFINITION

Hashing

Turning an object into a number (`hashCode()`) to decide which **bucket** it goes in, so it can be found fast.

```
add("Hita")
   |
   v
hashCode() --> bucket 5 --> is there an equals() match inside? --> yes: ignore / no: add

 bucket:    0        1        2          3        4       5
          [   ]    [   ]   [Somya]     [   ]    [   ]   [Hita] -> [Disha]
                                                          (collision: same bucket)
```

Our own class (`Student`) must override both:

Without `equals()` + `hashCode()`	With them (same id = same student)
Two `Hita` objects → size **2**	size **1**

*** REMEMBER

If two objects are `equals()`, their `hashCode()` **must** be the same.
`HashSet` uses a `HashMap` inside (elements are stored as keys).

26. TreeSet: Sorted Set

Scholarship cut-off 400. Totals: `[244, 412, 438, 465]`

Method	Meaning	Result
`first()` / `last()`	Smallest / biggest	`244` / `465`
`floor(450)`	Biggest ≤ 450	`438`
`ceiling(400)`	Smallest ≥ 400	`412`
`headSet(400)`	Below 400	`[244]`
`tailSet(400)`	400 and above	`[412, 438, 465]`

- Comes from `SortedSet` and `NavigableSet`.
- Sorts using `compareTo()` (`Comparable`) or a `Comparator`.

*** REMEMBER

`TreeSet` does **not** allow `null`.

27. Set Comparison

	`HashSet`	`LinkedHashSet`	`TreeSet`
Order	None	Insertion	Sorted
Speed	**Fastest** O(1)	Fast O(1)	Slower O(log n)
Duplicates checked by	`hashCode()` + `equals()`	`hashCode()` + `equals()`	`compareTo()` / `compare()`
`null`	One	One	**No**
Inside	`HashMap`	`LinkedHashMap`	`TreeMap`

PART C

Queue And Deque

28. Queue: Definition

DEFINITION

Queue

A collection that works as **FIFO** (First In, First Out). Add at the **back**, remove from the **front**.

Line at the fee counter:

```
front                          back
  |                             |
  v                             v
[Hita] [Sakshi] [Somya] [Ritika]   <-- offer() adds here
  ^
  poll() removes here
```

```
Queue<String> feeLine = new LinkedList<>();
feeLine.offer("Hita");
feeLine.offer("Sakshi");
feeLine.poll();             // Hita is served first
feeLine.peek();             // Sakshi (next, not removed)
```

29. Queue Methods: Two Sets

Job	Throws exception if it fails	Returns special value
Add at back	`add(x)`	`offer(x)` → `false`
Remove from front	`remove()` → `NoSuchElementException`	`poll()` → `null`
See the front	`element()` → `NoSuchElementException`	`peek()` → `null`

*** REMEMBER

Prefer **`offer`**, **`poll`**, **`peek`**: an empty queue does not crash the program.

30. PriorityQueue

DEFINITION

PriorityQueue

A queue where the element with the **highest priority** comes out first (natural order or a `Comparator`), not the one that came first.

Scholarship interview: highest total goes first.

```
PriorityQueue<Student> interview =
        new PriorityQueue<>((a, b) -> b.total - a.total);
// poll() one by one: Somya 465, Hita 438, Ritika 412, Sakshi 244
```

- Inside: a **binary heap** (min-heap by default: smallest first)
- `null` **not** allowed

*** REMEMBER

Printing a PriorityQueue may look **unsorted**. Only the **front** is guaranteed. Use `poll()` one by one.

31. Deque and ArrayDeque

Deque (double-ended queue, say "deck") A queue where you can add and remove at **both** ends. Works as a **queue** and as a **stack**.

DEFINITION

ArrayDeque

A Deque that uses a resizable **circular array**. Faster than `Stack` and `LinkedList`.

```
addFirst / pollFirst  <-->  [ Hita | Sakshi | Somya ]  <-->  addLast / pollLast
```

As a stack (LIFO)	As a queue (FIFO)
`push(x)` – add on top	`offerLast(x)` – add at back
`pop()` – remove from top	`pollFirst()` – remove from front
`peek()` – see top	`peekFirst()` – see front

*** REMEMBER

`ArrayDeque` is the best class for **both** stack and queue. No `null` allowed.

32. Queue Family Comparison

	`LinkedList`	`PriorityQueue`	`ArrayDeque`
Order out	FIFO	By priority	FIFO or LIFO
Inside	Linked nodes	Binary heap	Circular array
`null`	Allowed	No	No
Both ends?	Yes (Deque)	No	Yes (Deque)
Use for	Queue + list together	"Most important first"	Stack / queue (default)

PART D

Map

33. Map: Definition

DEFINITION

Map

An object that stores **key** → **value** pairs. **Keys are unique**; values can repeat.

DEFINITION

Entry

One key-value pair (`Map.Entry<K, V>`).

The office asks: "Show me roll number **103**." No loop needed.

```
Map<Integer, Student> register = new HashMap<>();
register.put(101, new Student(101, "Hita", 438));
register.put(103, new Student(103, "Somya", 465));
register.get(103);          // Somya, jumps straight there
```

```
 key        value
+-----+    +------------------+
| 101 | -> | Hita    438      |
| 103 | -> | Somya   465      |
+-----+    +------------------+
```

34. Map Methods

Method	Does	In our register
`put(k, v)`	Add; same key → value **replaced**	Sakshi re-checked: 244 → 310
`get(k)`	Value, or `null`	`get(999)` → `null`
`getOrDefault(k, d)`	Value, or d	Counting grades
`containsKey(k)` / `containsValue(v)`	Is it there?	`102` → `true`
`remove(k)`	Remove the pair	Disha leaves
`putIfAbsent(k, v)`	Put only if key is free	101 taken → no change
`keySet()`	All keys (a Set)	`[101, 102, 103]`
`values()`	All values (a Collection)	all students
`entrySet()`	All pairs (a Set of Entry)	loop below

```
for (Map.Entry<Integer, Student> e : register.entrySet()) {
    System.out.println(e.getKey() + " -> " + e.getValue().name);
}
```

35. How HashMap Works Inside

```
put(103, Somya)
      |
  103.hashCode() --> bucket index
      |
 bucket: 0      1      2       3              ...  15
       [   ]  [   ]  [101]  [103] -> [119]   ...  [ ]
                              collision: kept in a list (tree if long)
```

1. **Hashing:** `key.hashCode()` → picks a **bucket** (default **16** buckets).
2. **Collision:** two keys in one bucket → small linked list (becomes a **tree** after 8).
3. `equals()` finds the right key inside the bucket.
4. **Load factor 0.75:** when 75% full (12 of 16) ...
5. **Rehashing:** ... buckets **double** (32) and every entry moves.

> *** REMEMBER
> HashMap = **hashCode()** → **bucket**, **equals()** → **key**. Default 16 buckets, load factor 0.75.

36. HashMap, LinkedHashMap, TreeMap, Hashtable

	`HashMap`	`LinkedHashMap`	`TreeMap`	`Hashtable`
Order	None	Insertion	**Sorted keys**	None
Inside	Hash table	Hash table + links	Red-black tree	Hash table
`null` key	One	One	**No**	**No**
`null` value	Yes	Yes	Yes	**No**
Synchronised	No	No	No	**Yes** (legacy)
Speed	O(1)	O(1)	O(log n)	O(1), slower (locks)
Use	**Default**	Keep order	Sorted keys	Old code only

- `TreeMap` extras: `firstKey()`, `lastKey()`, `headMap(k)`, `tailMap(k)`
- Thread-safe today: `ConcurrentHashMap`, not `Hashtable`

> *** REMEMBER
> A key class **must** override `equals()` and `hashCode()`, and a key should **never change** after `put`.

PART E

Working With Collections

37. Iterator: Visiting Elements

DEFINITION

Iterator

An object that visits the elements of a collection one by one, and can **safely remove** the current one.

Scholarship list: remove everyone below 400.

```
Iterator<Student> it = list.iterator();
while (it.hasNext()) {
    Student s = it.next();
    if (s.total < 400) {
        it.remove();          // ✅ safe
    }
}
```

Iterator	`ListIterator` (List only)	`Enumeration` (legacy)
`hasNext()`, `next()`, `remove()`	+ `hasPrevious()`, `previous()`, `set()`, `add()`	`hasMoreElements()`, `nextElement()`
Forward only	**Both ways**	Forward, **cannot remove**
All collections	`List`	`Vector`, `Hashtable`

38. Fail-Fast vs Fail-Safe

```
for (Student s : list) {
    if (s.total < 400) list.remove(s);    // ❌ ConcurrentModificationException
}
```

	Fail-fast	Fail-safe
Changed during loop	Throws `ConcurrentModificationException`	No exception
Works on	The original collection	A **copy**
Examples	`ArrayList`, `HashMap`, `HashSet`	`CopyOnWriteArrayList`, `ConcurrentHashMap`

Ways to loop

Loop	Can remove?
`for (int i...)` + `get(i)`	Risky (index shifts)
for-each	**No** → exception
`list.forEach(s -> ...)`	No
`Iterator`	**Yes**: `it.remove()`
`list.removeIf(s -> s.total < 400)`	**Yes**, short form

***** REMEMBER**

Never remove inside for-each. Use `it.remove()` or `removeIf()`.

39. Sorting: Comparable vs Comparator

Comparable – the class gives its **one natural order** with `compareTo()`. **Comparator** – a separate object gives **any order** with `compare()`.

```
class Student implements Comparable<Student> {
    public int compareTo(Student o) { return this.id - o.id; }   // by roll no
}
Collections.sort(list);                                          // roll order
list.sort(Comparator.comparing((Student s) -> s.total)
        .reversed()
        .thenComparing(s -> s.name));                            // merit list
```

Merit list: `Somya 465, Disha 438, Hita 438, Ritika 412, Sakshi 244`

	`Comparable`	`Comparator`
Package	`java.lang`	`java.util`
Method	`compareTo(T o)`	`compare(T a, T b)`
Written	Inside the class	Outside the class
Orders	One	Many

Result	Meaning
Negative	First comes **before**
0	Same place
Positive	First comes **after**

40. The `Collections` Utility Class

DEFINITION

Collections

A utility class with only **static** methods that work on collections.

`totals = [438, 244, 465, 412, 438]`

Method	Result
`sort(list)`	`[244, 412, 438, 438, 465]`
`reverse(list)`	Reverse order
`shuffle(list)`	Random viva order
`max(list)` / `min(list)`	`465` / `244`
`frequency(list, 438)`	`2`
`binarySearch(list, 412)`	`1` (list **must be sorted**)
`unmodifiableList(list)`	Read-only view (notice board)
`synchronizedList(list)`	Thread-safe wrapper

`Collection` vs `Collections`

`Collection`	`Collections`
Interface	**Utility class**
Parent of List, Set, Queue	Only `static` helpers
`section.add(x)`	`Collections.sort(section)`

*** REMEMBER

One letter, two very different things.

41. Immutable Collections

```
List<String> subjects = List.of("Java", "DBMS", "Maths", "English", "Networks");
Set<String> days = Set.of("Mon", "Wed", "Fri");
Map<Integer, String> rooms = Map.of(101, "Lab 1", 102, "Lab 2");
// subjects.add("Python");   // ✗ UnsupportedOperationException
```

`List.of()`	`Collections.unmodifiableList(x)`
Fixed from the start	A read-only **view** of another list
No `null` allowed	Changes in the original show through

PART F

The Full Picture

42. Master Table: Every Class at a Glance

Class	Interface	Order	Duplicates	`null`	Synchronised	Inside
`ArrayList`	List	Insertion	Yes	Yes	No	Dynamic array
`LinkedList`	List, Deque	Insertion	Yes	Yes	No	Doubly linked list
`Vector`	List	Insertion	Yes	Yes	**Yes**	Dynamic array
`Stack`	List	LIFO	Yes	Yes	**Yes**	Vector
`HashSet`	Set	None	**No**	One	No	HashMap
`LinkedHashSet`	Set	Insertion	**No**	One	No	LinkedHashMap
`TreeSet`	NavigableSet	Sorted	**No**	No	No	TreeMap
`PriorityQueue`	Queue	Priority	Yes	No	No	Binary heap
`ArrayDeque`	Deque	FIFO / LIFO	Yes	No	No	Circular array
`HashMap`	Map	None	Keys no	1 key	No	Hash table
`LinkedHashMap`	Map	Insertion	Keys no	1 key	No	Hash table + links
`TreeMap`	NavigableMap	Sorted keys	Keys no	No key	No	Red-black tree
`Hashtable`	Map	None	Keys no	No	**Yes**	Hash table

43. Speed Table (Big-O)

Operation	`ArrayList`	`LinkedList`	`HashSet` / `HashMap`	`TreeSet` / `TreeMap`
Add at end	O(1)	O(1)	O(1)	O(log n)
Get by index	**O(1)**	O(n)	–	–
Search (`contains`)	O(n)	O(n)	**O(1)**	O(log n)
Remove from start	O(n)	**O(1)**	O(1)	O(log n)

- **O(1)** = same time no matter how many students
- **O(log n)** = a little slower as it grows
- **O(n)** = has to look at every student

44. Legacy Classes

Legacy (Java 1.0)	Modern replacement
`Vector`	`ArrayList`
`Stack`	`ArrayDeque`
`Hashtable`	`HashMap` / `ConcurrentHashMap`
`Enumeration`	`Iterator`

*** REMEMBER

Legacy classes are **synchronised** (slow). Know them for exams and old code; use the modern ones in new code.

45. Which Collection Do I Use? (Decision Map)

```
Size never changes?              → Array

Need order + index + duplicates? → List
      mostly reading             → ArrayList
      add/remove at the ends     → LinkedList

No duplicates?                   → Set
      fastest                    → HashSet
      keep insertion order       → LinkedHashSet
      always sorted              → TreeSet

Process one by one?              → Queue / Deque
      first come, first served   → ArrayDeque (queue)
      last in, first out         → ArrayDeque (stack)
      most important first       → PriorityQueue

Find by a key?                   → Map
      fastest                    → HashMap
      keep insertion order       → LinkedHashMap
      sorted keys                → TreeMap
```

Need	Pick
Section register	`ArrayList`
Workshop sign-up, no repeats	`HashSet` / `LinkedHashSet`
Merit list always sorted	`TreeSet` / `TreeMap`
Fee counter line	`ArrayDeque` (Queue)
Answer-sheet pile	`ArrayDeque` (stack)
Interview by highest marks	`PriorityQueue`
Roll no → student	`HashMap`

46. Top Remember Points (***)

1. *** Collections store **objects only**; all in `java.util`.
2. *** `Map` is **not** a `Collection`.
3. *** List = order + duplicates + index. Set = no duplicates. Queue = FIFO. Map = key → value.
4. *** `ArrayList` grows **1.5x**, `Vector` grows **2x**; both start at **10**.
5. *** `ArrayList` = fast read. `LinkedList` = fast add/remove at ends.
6. *** `Vector`, `Stack`, `Hashtable` are **legacy + synchronised**.
7. *** `Stack` extends `Vector`; use `ArrayDeque` for stacks today.
8. *** `HashSet` / `HashMap` need **`equals()`** + **`hashCode()`** for our own classes.
9. *** `HashMap`: 16 buckets, load factor 0.75, allows one `null` key; `Hashtable` allows none.
10. *** `TreeSet` / `TreeMap`: sorted, **no `null`**.
11. *** Never remove inside for-each → use `Iterator.remove()` or `removeIf()`.
12. *** `Comparable` = one natural order (inside). `Comparator` = many orders (outside).
13. *** `Collection` = interface, `Collections` = utility class.

47. Quick Check

YOUR TURN

Which collection would you use?

1. All students of CSE-A in admission order.
2. Students in the Java workshop (no one twice), in sign-up order.
3. Students standing at the fee counter.
4. Find a student's details from the roll number.
5. Scholarship interview: highest total goes first.
6. Mr. Verma's pile of answer sheets.
7. Roll numbers always kept sorted.
8. The 5 subject marks of one student.

ANSWERS

1. `ArrayList`
2. `LinkedHashSet`
3. `Queue` → `ArrayDeque` / `LinkedList`
4. `HashMap<Integer, Student>`
5. `PriorityQueue` with a Comparator
6. Stack → `ArrayDeque` (`push` / `pop`)
7. `TreeSet<Integer>` (or `TreeMap` for roll no → student)
8. Array `int[5]` (size never changes)

Thank You

KEY TAKEAWAY

Array → fixed box.
Collection → smart container that grows, searches, sorts and stops duplicates for you.

Choose the container by the job.